APPENDIX A
ARTIFACT APPENDIX

This appendix describes how to reproduce the results in the main body of the paper and is written to be read on its own. The artifact is a complete prototype of *OrionLink*, a privacy-preserving single sign-on (SSO) system. In brokered SSO a user logs in to a website, the relying party (RP), through an intermediary, the broker, which forwards the login to the identity provider (IdP) that authenticates the user and issues the tokens. In *OrionLink* the broker is *oblivious*: it routes logins without learning the user's cross-RP identity or any personal data. The artifact contains the four deployable roles of the paper's Implementation section (Section V-A): IdP, broker, RP, and the Trusted Computation Agent (TCA), a browser-side script on its own origin that performs the user's client-side cryptography. It implements the four protocol phases (Registration, Login Request, User Authentication and Consent, Login Response), the two extensions (private authorization management, forward-secure token refresh), and one measurement harness per experiment of the Evaluation section (Section V-B); about 9K lines of Go and JavaScript in total. "README §x" below refers to `README.md` at the repository root.

A. Description & Requirements

1) How to access: The artifact repository is available via a persistent DOI: https://doi.org/10.5281/zenodo.22815552, and on GitHub: https://github.com/Ooo0ption/Orionlink. The four roles are also published as one prebuilt Docker image, `walnutd/orionlink-ae:ndss2027-v2`.

2) Hardware dependencies: No special hardware. A commodity x86-64 machine with 8 cores, 16 GB of RAM and about 10 GB of free disk runs every experiment and is the basis for the time estimates below. The paper's results were produced on an AMD Ryzen 9 9950X3D (16 cores) with 96 GB of RAM under Ubuntu 24.04.

3) Software dependencies: Two layers. (i) To deploy the prototype and to run E3 the host needs only Docker $\geq$ 24 with the Compose v2 plugin, on Linux (Ubuntu 24.04 recommended) or on Windows with Docker Desktop in Linux-container mode; every command below has a PowerShell equivalent in the README. The image is self-contained and resolves no package from the network at run time. (ii) E1 and E2 are host-side harnesses that drive the containers from outside, so the host additionally needs Go $\geq$ 1.25 (E1 and the unit tests), Node.js $\geq$ 18 (E2, which installs its own Chromium) and Python $\geq$ 3.8 (E2, standard library only). Install commands for Ubuntu, macOS and Windows, with version checks, are in README §3.2; `requirements.txt` is the authoritative list. Nothing is ever installed inside a container.

B. Artifact Installation & Configuration

README §3.1 gives the whole reproduction, from checkout to the three result sets, as one numbered command sequence; the steps here and the experiments below expand its rows. Every command is typed in the host shell at the repository root.

1) **Pull the image** (README §2.2). Set `ORION_IMAGE=walnutd/orionlink-ae:ndss2027-v2` in the shell and run `docker image pull`; the README shows the expected status line. To build the same image from source instead, online or fully offline, follow README §5 and continue here.
2) **Start the containers** (README §2.3). `docker compose up -d --wait` starts the roles in dependency order and returns once all four report healthy; ports 3000–3003 must be free. The README lists the four addresses and marks the only one to open by hand, the RP at `http://localhost:3002/ssso/`.
3) **Install the host-side tools** (README §3.2): `go mod download`, then `npm ci` and `npx playwright install chromium` in `benchmark/e2/browser`, each once.

The stack runs in one of two profiles, selected by `ORION_PROFILE`. The default, `demo`, is the deployment to click through by hand. `bench` runs the same binary and pages but registers the RP at startup, seeds synthetic users and mounts the load-test endpoints; E2 and E3 require it.

Optionally, the front end of the prototype lets the evaluator walk through the *OrionLink* flow one button at a time (README §4), registering the RP, signing in as the demo user, refreshing the token and revoking the authorization. This walk-through is *not* a prerequisite for any experiment.

C. Major Claims

The major claims of the paper that this artifact substantiates are summarized as follows.

- (C1): *OrionLink*'s privacy mechanisms incur only marginal local computation, and the anonymous authenticated key agreement AnonyAKA (AAKA), which RP and IdP run through the broker, is the most expensive stage on the login path. This claim is substantiated by Experiment (E1), which reproduces Table II.
- (C2): End-to-end browser login remains within interactive bounds and degrades gracefully with network delay, growing approximately linearly with the network round-trip time (RTT) for both *OrionLink* and the standard brokered-SSO baseline. This claim is substantiated by Experiment (E2), which reproduces Table III and Fig. 7.
- (C3): The system stays low-latency under sustained load on its most expensive interface, the AnonyAKA path jointly executed by RP, broker and IdP. This claim is substantiated by Experiment (E3), which reproduces Fig. 8.

D. Evaluation

One experiment per claim. Each is driven by a single runner under `benchmark/` and writes its report to `benchmark/results/`; iteration counts and arrival rates are arguments, so an experiment runs at the paper's setting or at a reduced

one. All commands are typed in the host shell at the repository root, and the evaluator never has to enter a container. The two steps that do run inside one, the k6 load generator of E3 and the netem delay setting, are started from the host by Compose.

Experiment (E1) *Local computation overhead* [Table II] (≤10 compute-min; README §3.3). Supports claim C1. It measures each privacy-enhancing stage in isolation and prints the six rows of Table II.

[Configuration Parameters] The paper's setting is 1,000 runs per stage; the argument defaults to 100. Every stage is measured in-process.

[Preparation and Execution] Needs Go on the host (step 3). The stack must be *down* (`docker compose down`): it would compete for CPU, and the unit tests bind port 3000 themselves.

```
go test ./benchmark/unit/
./benchmark/run-e1.sh 1000
```

[Results] The mean time per operation for the six stages, plus their total. The overhead stays small at every stage, which supports the claim that the privacy mechanisms of *OrionLink* add negligible local computation. Absolute values scale with single-core speed, so a deviation of up to a factor of two is expected on slower hardware.

Experiment (E2) *End-to-end latency* [Table III, Fig. 7] (≤45 compute-min; README §3.4). Supports claim C2. A Playwright browser-automation harness drives the running containers and produces every row of Table III: the *SSO Flow* rows from a full login, the *Authorization management* row from the PIN-based decryption in the TCA, and the *Token refresh* row as an authenticated browser request that takes the live RP → broker → IdP path and back.

[Configuration Parameters] The paper's setting is 100 runs per configuration, at emulated RTTs of 0, 40, 100 and 200 ms.

[Preparation and Execution] Needs Node.js and Python on the host (step 3). Start the stack in the bench profile, then run the harness; repeat at each RTT after applying the per-direction delay with `./netem-compose.sh set-egress <delay>` (README §3.6).

```
ORION_PROFILE=bench docker compose up -d --wait
ORION_PROFILE=bench ./benchmark/run-e2.sh 100
```

The brokered OpenID Connect (OIDC) baseline that Table III compares against is a separate stack with its own image and ports, documented in `benchmark/oidc-baseline/`.

[Results] The runner ends by printing Table III (`benchmark/e2/summarize.py` re-renders it from the reports). Browser timings are noisy, so accept a deviation of roughly 50% on absolute values. The claim to check is that the *OrionLink* total at 40 ms RTT is on the order of two seconds and grows linearly with the RTT.

Experiment (E3) *Stress test of the* AnonyAKA *interface* [Fig. 8] (≤20 compute-min; README §3.5). Supports claim C3. It drives the AAKA work of the three roles under a rising arrival rate and produces the three lines of Fig. 8, one per role.

[Configuration Parameters] The paper's setting is an offered load of 100, 500, 1,000, 1,300 and 1,500 requests per second (RPS), generated by k6 with `constant-arrival-rate`, 60 s per rate, for each of the three roles. By default the runner starts a lightweight configuration, 10 and 200 RPS, that any machine sustains; the paper's ladder is requested with `--rps`.

[Preparation and Execution] Needs only Docker. Start the stack in the bench profile with `ORION_NETEM_DELAY` empty, then start the load generator, a fifth Compose service that carries the pinned k6 binary and the E3 scripts inside the submitted image. The command is typed on the host; Compose creates and removes the container.

```
docker compose --profile benchmark run --rm e3
docker compose --profile benchmark run --rm e3 \
    --rps 100,500,1000,1300,1500
```

The first line is the lightweight default (about eight minutes; `--quick` shortens it to two); the second reproduces the paper's ladder. Anything after `e3` is passed to `benchmark/run-e3.sh`.

[Results] One JSON report per role with, for each rate, the target and achieved rate, the p50/p95/p99 latency, the server-side p95 and the dropped-request count; a rate is a valid data point only when nothing was dropped and the achieved rate matches the target. Plot p95 against the rate to obtain Fig. 8. The result to check is the paper's: at 1,000 RPS the p95 latency stays at about 10 ms for the IdP and below that for RP and broker, i.e., flat across the first three rates. Above 1,000 RPS the IdP line rises steeply as its pairing-bound AAKA work saturates the CPU; on a machine with fewer cores than the paper's this knee moves to a lower rate, which does not affect the claim.

E. Customization

The deployment is configured through `.env.example`: `ORION_IMAGE` selects the image, `ORION_PROFILE` the profile, `ORION_NETEM_DELAY` the egress delay, and `ORION_*_URL` / `ORION_*_PORT` relocate the roles. `benchmark/README.md` documents every remaining parameter and the troubleshooting steps.

F. Notes

The prototype is dominated by public-key cryptography and is bound to the CPU, so every experiment is sensitive to other load on the machine and the tolerances quoted above need not cover every deployment; close other workloads first. In (E3) the load generator shares the cores with the server roles and every request crosses the Docker network stack, so running the roles and k6 directly on the host sustains a higher rate.